S.A. ENGINEERING COLLEGE

(An Autonomous –Institute Level Research Center- Affiliated to Anna University)

Accredited by NBA, NAAC with 'A' Grade & ISO 9001 : 2015 Certified Institution

POONAMALLE – AVADI MAIN ROAD,

THIRUVERKADU, CHENNAI – 600 077.

RECORD OF LABORATORY WORK

Subject.....

Name.....

Reg No.....

Branch.....

Year..... Semester.....

S.A. ENGINEERING COLLEGE

(An Autonomous –Institute Level Research Center- Affiliated to Anna University)

Accredited by NBA, NAAC with 'A' Grade & ISO 9001 : 2015 Certified Institution

POONAMALLE – AVADI MAIN ROAD,
THIRUVERKADU, CHENNAI – 600 077.

CERTIFICATE

Name.....

Year.....Semester.....Branch.....

University Register No.

Certified that this is the Bonafide record of the work done by the above student in the.....during the year 20 - 20

Signature of the Head of the Department Signature of Course Lecturer

Submitted for the University Practical Examination held on.....

Of **S.A. ENGINEERING COLLEGE, POONAMALLE – AVADI MAIN ROAD, THIRUVERKADU, CHENNAI – 600 077**

Internal Examiner External Examiner

S.A. ENGINEERING COLLEGE- CHENNAI-77

(An Autonomous Institution Level Research Center Affiliated to Anna University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

VISION OF THE DEPARTMENT

To pioneer in the emerging areas of Computer Science and Engineering education and research, fostering innovation, ethical values, sustainable technologies that advance the digital world and improve societal well-being.

MISSION OF THE DEPARTMENT

1. Educational Excellence:

To provide a transformative learning experience that integrates theory with practical skills, fostering creativity, critical thinking, and lifelong learning in Computer Science and Engineering.

2. Innovative Research:

To promote research and interdisciplinary collaboration, addressing contemporary challenges in technology and society.

3. Sustainable and Ethical Practices:

To facilitate the development of sustainable technological solutions with ethical practices that positively impact the environment and society.

4. Industry and Society:

To build partnerships with industry and stakeholders, enhancing the employability and entrepreneurial spirit of students, while contributing to technological and societal advancement.

S.A. ENGINEERING COLLEGE- CHENNAI-77

(An Autonomous Institution Level Research Center Affiliated to Anna University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

PROGRAMME EDUCATIONAL OBJECTIVES

PEO 1: Graduates shall have professional competency in the field of Computer Science and Engineering for pursuing higher education, research or as entrepreneurs.

PEO 2: Graduates shall work in a business environment with ethical standards, leadership qualities and communication necessary for engineering principles.

PEO 3: Graduates shall adapt to emerging technologies and respond to the challenges of the environment and society forever.

PROGRAMME SPECIFIC OUTCOMES

PSO 1: To analyze, design and develop computing solutions by applying foundational concepts of Computer Science and Engineering.

PSO 2: To apply software engineering principles and practices for developing quality software for scientific and business applications.

PSO 3: To adapt to emerging Information and Communication Technologies (ICT) to innovate ideas and solutions to existing/novel problems.

PROGRAMME OUTCOMES

Engineering Graduates will be able to:

1. **Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.**

2. **Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.**

3. **Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.**

4. **Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.**

5. **Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modelling to complex engineering activities with an understanding of the limitations.**

6. **The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.**

7. **Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.**

8. **Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.**

9. **Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.**

10. **Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.**

11. **Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader**

in a team, to manage projects and in multidisciplinary environments.

12. Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

CS3203	PROGRAMMING IN PYTHON LABORATORY	L	T	P	C
		0	0	4	2

COURSE OBJECTIVES:

- To study python programs with conditionals and loops
- To use functions for python structured programs.
- Use strings for structuring Python programs.
- Represent compound data using Python lists, tuples and dictionaries.
- To read and write data from and to files in python.

LIST OF EXPERIMENTS:

1. Write a program to display the largest number among three numbers.
2. Write a program to display the Fibonacci series by using looping constructs.
3. Write a function to compute the GCD of two numbers.
4. Explore String Functions
5. With the help of strings, array or list, display a simple calendar in python program without using the calendar module.
6. With the help of list perform linear search and Binary search.
7. Create a text file using python file I/O. Read the content of the file and change them from lower to upper case characters.
8. Programs that take command line arguments (word count)
9. Find the most frequent words in a text read from a file.
10. Write a program using Dictionaries to compute the following.
    i. Students marks statement
    ii. Retail bill preparation
11. Write a program using sets to compute the following.
    i. Intersection,
    ii. Union,
    iii. Symmetric Difference between sets.
12. Write a program using exceptions to calculate the following.
    i. Voter's age validation,
    ii. Marks range validation (0-100).
13. Write a program to Display student information using class and object.
14. Write a program to Deposit or Withdraw money in a bank account using class and objects

LIST OF EQUIPMENT FOR A BATCH OF 30 STUDENTS:

HARDWARE: 30 terminals
SOFTWARE: Python 3 interpreter for Windows/Linux

TOTAL: 60 PERIODS

COURSE OUTCOMES:

At the end of the course the student should be able to

- Design simple programs using conditionals and loops.
- Write functions to solve mathematical problems
- Use strings for structuring Python programs.
- Represent compound data using Python lists, tuples and dictionaries.
- Identify to read and write data from and to files in python.

S.A. ENGINEERING COLLEGE- CHENNAI-77

(Autonomous- Institute Level Research Centre, Affiliated to Anna University)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Course Objectives:

- To study python programs with conditionals and loops
- To use functions for python structured programs.
- Use strings for structuring Python programs.
- Represent compound data using Python lists, tuples and dictionaries.
- To read and write data from and to files in python.

Course Outcomes:

- Design simple programs using conditionals and loops.
- Write functions to solve mathematical problems
- Use strings for structuring Python programs.
- Represent compound data using Python lists, tuples and dictionaries.
- Identify to read and write data from and to files in python.

CO"s-PO"s and PSO"s MAPPING:

CO's PO's and PSO's MAPPING													
CO/ PO/ PSO	**PO1**	**PO2**	**PO3**	**PO4**	**PO5**	**PO6**	**PO7**	**PO8**	**PO9**	**PO10**	**PO11**	**PSO1**	**PSO2**
CO1	3	2	1	-	1	-	-	-	1	2	-	3	2
CO2	3	3	2	1	1	-	-	-	1	2	-	3	2
CO3	3	3	2	1	1	-	-	1	1	2	-	3	3
CO4	3	3	2	1	2	-	-	-	1	2	-	3	3
CO5	2	3	3	-	1	1	-	2	2	2	1	3	3

1 - Low, 2 - Medium, 3 - High, – no correlation

.

INDEX

SUBJECT CODE: CS3203

SUBJECT NAME: PROGRAMMING IN PYTHON LABORATORY

SI.NO	DATE	NAME OF THE EXPERIMENT	MAPPED CO'S	Marks	Staff sign
1		WRITE A PROGRAM TO DISPLAY THE LARGEST NUMBER AMONG THREE NUMBERS.	CO1		
2		WRITE A PROGRAM TO DISPLAY THE FIBONACCI SERIES BY USING LOOPING CONSTRUCTS.	CO2		
3		WRITE A FUNCTION TO COMPUTE THE GCD OF TWO NUMBERS.	CO2		
4		EXPLORE STRING FUNCTIONS	CO1		
5		WITH THE HELP OF STRINGS, ARRAY OR LIST, DISPLAY A SIMPLE CALENDAR IN PYTHON PROGRAM WITHOUT USING THE CALENDAR MODULE.	CO3		
6		WITH THE HELP OF LIST PERFORM LINEAR SEARCH AND BINARY SEARCH.	CO3		
7		CREATE A TEXT FILE USING PYTHON FILE I/O. READ THE CONTENT OF THE FILE AND CHANGE THEM FROM LOWER TO UPPER CASE CHARACTERS.	CO4		
8		PROGRAMS THAT TAKE COMMAND LINE ARGUMENTS (WORD COUNT)	CO4		
9		FIND THE MOST FREQUENT WORDS IN A TEXT READ FROM A FILE.	CO4		
10		WRITE A PROGRAM USING DICTIONARIES TO COMPUTE THE FOLLOWING. i. STUDENTS MARKS STATEMENT ii. RETAIL BILL PREPARATION	CO3		
11		WRITE A PROGRAM USING SETS TO COMPUTE THE FOLLOWING. i. INTERSECTION, ii. UNION, iii. SYMMETRIC DIFFERENCE BETWEEN SETS.	CO3		
12		WRITE A PROGRAM USING EXCEPTIONS TO CALCULATE THE FOLLOWING. i. VOTER'S AGE VALIDATION, ii. MARKS RANGE VALIDATION (0-100).	CO4		

13		WRITE A PROGRAM TO DISPLAY STUDENT INFORMATION USING CLASS AND OBJECT.	CO5		
14		WRITE A PROGRAM TO DEPOSIT OR WITHDRAW MONEY IN A BANK ACCOUNT USING CLASS AND OBJECTS	CO5		

SIGNATURE OF THE FACULTY

EX NO: 1 **GREATEST OF THREE NUMBERS**

AIM:

To write a program to find the greatest of three number using python.

ALGORITHM:

Step 1: Start

Step 2: Input three numbers: a, b, c.

Step 3: Check:If a > b and a > c: print “a is the greatest”.

Step 4: Else if b > a and b > c: print “b is the greatest”. Else: print “ c is the greatest”.

Step 5: End

PROGRAM:

```
a=int(input("Enter the First number:"))
b=int(input("Enter the Second number:"))
c=int(input("Enter the Third number:"))
if (a>b)&(a>c):
   print("%d  is Greater" %(a))
elif (b>a)&(b>c):
   print("%d  is Greater" %(b))
else:
   print("%d  is Greater" %(c))
```

OUTPUT:

Enter the First number: 5

Enter the Second number: 4

Enter the Third number: 7

7 is Greater

RESULT:

Thus the program to find the greatest of three numbers is written and executed successfully.

EX NO: 2 **FIBONACCI SERIES BY USING LOOPING CONSTRUCTS**

AIM:

To display the Fibonacci series by using looping constructs

ALGORITHM:

Step 1: Start

Step 2: Input the number of terms, n.

Step 3: Initialize variables:

f = 0

f1 = 0

f2 = 1

Step 4: Print f1 and f2.

Step 5: Repeat the following steps for i = 3 to n:

Compute f = f1 + f2.

Print f.

Update values:f1 = f2 and f2 = f

Step 6: End the loop.

Step 7: End

Program

```
f=0
f1=0
f2=1
n=int(input("Enter the Range:"))
print(f1,f2,end=" ")
f=f1+f2
while f<n:
    f=f1+f2
    f1=f2
    f2=f
    print(f,end=" ")
```

Output:

Enter First Range: 5

0 1 1 2 3 5

RESULT:

Thus the Fibonacci series using looping constructs is written and executed successfully

EX NO: 3 **GCD OF TWO NUMBERS USING FUNCTION**

AIM:

To find the GCD of two numbers using Function.

ALGORITHM:

Step 1: Start

Step 2: Define a function GCD(a, b):

Input: Two integers a and b.

Repeat while b ≠ 0:

Compute the remainder: remainder = a % b.

Update a = b and b = remainder.

Return a as the GCD.

Step :3 End Function

Step 4: Main Program:

Input two integers a and b.

Call the function GCD(a, b) and store the result.

Output the result as the GCD of a and b.

Step 5: End

Program

```
def gcd(a,b):
   if(b==0):
      return a
   else:
      return gcd(b,a%b)
a=int(input("Enter First Number:"))
b=int(input("Enter Second Number:"))
GCD=gcd(a,b)
print("GCD is:",GCD)
```

Output:

```
Enter First Number: 30
Enter second Number: 15
GCD is: 15
```

RESULT: Thus the GCD of two numbers program is written and executed successfully.

EX NO: 4 **EXPLORE STRING FUNCTIONS**

AIM:

To implement String Functions and Methods.

ALGORITHM:

Step 1: Start

Step 2: Read the String

Step 3: Implement the String Functions and Methods

Step 4: Print String

Step5: Stop

Program:

1. Getting the Length of a String

The len() function is used to determine the size of a string that is, the number of characters in a string.

Program:

```
str="programming"
print(len(str))
```

Output:

11

2. Str.Capitaize()

This method will return a copy of the given string with the first character in uppercase and remaining characters in lower case format.

Program:

```
str="programming"
print(str.capitalize())
```

Output:

Programming

3. Str.Upper()

This will return a copy of the given string with all the characters in Uppercase.

Program:

```
str="programming"
print(str.upper())
```

Output:

PROGRAMMING

4. Str.lower()

This will return a copy of the given string with all the characters in Lower case.

Program:

```
str="PROGRAMMING"
print(str.lower())
```

Output:
programming

5. str.count()

This method will return the total number of overlapping occurrences of the given substring with the given range of [start,end].

Program:

```
str="String string String string String"
print(str.count("String"))
print(str.count("string"))
```

Output:

3

2

6. str.find()

This method will return the lowest index of the given string which given the range of [start,end].

Program:

```
str="String string String string String"
print(str.find("string"))
```

Output:

7

7. str.isalnum()

This method will return the Boolean Value True, if there is one character at least and one numeric value with no space otherwise it will return False.

Program:

```
str1="String string String string String3"
str2="StringstringStringstringString3"
print(str1.isalnum())
print(str2.isalnum())
```

Output

False

True

8. Str.isalpha()

This method will return the Boolean Value True, if there is one character at least and if all the all the characters in the string are alphabetic otherwise it will return False.

Program:

```
str1="Python3"
str2="Python"
print(str1.isalpha())
print(str2.isalpha())
```

Output

False

True

9. str.isdigit()

This method will return the Boolean Value True, if there is one character at least and if all the characters in the given string are Numeric Value otherwise it will return False.

Program:

```
str1="Python"
str2="3"
print(str1.isdigit())
print(str2.isdigit())
```

Output

False

True

10. str.islower()

This Method will return a Boolean Value True in a case, where all the characters present in the string are lower case otherwise it will return False.

Program:

```
str1="Python"
print(str1.islower())
```

Output

False

11. str.isupper()

This method will return a Boolean Value True in a case, where all the characters present in the string are uppercase otherwise it will return False.

Program:

```
str1="PYTHON"
print(str1.islower())
```

Output

True

12.str.title()

This method will return a Title Case Version of the given string, where the first letter of a word is in uppercase and the rest of the characters in the lower case.

Program:

```
str1="problem solving and python programming"
print(str1.title())
```

Output

Problem Solving And Python Programming

13. str.isspace()

This method will return True, if the given string contains only Whitespace otherwise it will return False.

Program:

```
str1=" "
print(str1.isspace())
```

Output:

```
True
```

14. str.split()

This method will return a list of all the words in the string by using the split() operator. It can split by using single quotes or double quotes.

Program:

```
str1="Problem Solving Python Programming"
print(str1.split( ' '))
```

Output:

```
['Problem', 'Solving', 'Python', 'Programming']
```

RESULT:

Thus the String Functions and Methods program are executed successfully.

EX NO: 5 **TO DISPLAY A SIMPLE CALENDAR WITHOUT USING CALENDAR MODULE IN PYTHON**

AIM:

To display a simple calendar without using calendar module in python.

ALGORITHM:

Step 1: Start

Step 2: Input the year (year) and month (month) from the user.

Step 3: Define arrays/lists:

Days of the week: days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]

Days in each month: month_days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

Step 4: Check for a leap year:

If year % 4 == 0 and (year % 100 != 0 or year % 400 == 0), update month_days[1] = 29 (February has 29 days in a leap year).

Step 5: Determine the first day of the given month:

If month < 3, set month += 12 and year -= 1.

Compute:

day_code = (1 + 2 * month + (3 * (month + 1)) // 5 + year + year // 4 - year // 100 + year // 400) % 7

Step 6: Output the calendar:

Print the month and year header (e.g., Month Year).

Print the days of the week using the days list.

Initialize a counter for indentation (day_code).

For each day in the range 1 to month_days[month - 1]:

Print each day number, aligning with its corresponding day of the week.

Start a new line after printing Saturday.

Step 7: End

Program

```
def is_leap_year(year):
    return year % 4 == 0 and (year % 100 != 0 or year % 400 == 0)



def get_first_day_of_month(year, month):
    if month < 3:
        month += 12
```

```
        year -= 1
    k = year % 100
    j = year // 100
    day_code = (1 + (13 * (month + 1)) // 5 + k + (k // 4) + (j // 4) - (2 * j)) % 7
    return (day_code + 6) % 7  # Adjusting to make Sunday = 0


def display_calendar(year, month):
    """Display the calendar for a given year and month."""
    days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
    month_days = [31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31]

    # Adjust February for leap years
    if is_leap_year(year):
        month_days[1] = 29

    # Get the number of days in the month and the first day of the month
    days_in_month = month_days[month - 1]
    first_day = get_first_day_of_month(year, month)

    # Print the calendar header
    print(f"\n   {year}-{month:02}")
    print(" ".join(days))

    # Print leading spaces for the first day
    print("    " * first_day, end="")

    # Print the days of the month
    for day in range(1, days_in_month + 1):
        print(f"{day:3} ", end="")
        if (first_day + day) % 7 == 0:  # New line after Saturday
            print()
    print()  # Final newline for formatting


# Input year and month
year = int(input("Enter the year: "))
```

```
month = int(input("Enter the month (1-12): "))

# Display the calendar
display_calendar(year, month)
```

Output:

```
Enter the year: 2025
Enter the month (1-12): 1

   2025-01
Sun Mon Tue Wed Thu Fri Sat
                  1   2   3   4
  5   6   7   8   9  10  11
 12  13  14  15  16  17  18
 19  20  21  22  23  24  25
 26  27  28  29  30  31
```

RESULT:

Thus the program to display Simple Calendar in python program without using the calendar module was executed successfully.

EX NO: 6 A **LINEAR SEARCH**

AIM:

Write a program to search an element in a list using linear search concept in python.

ALGORITHM:

Step 1: Start

Step 2: Input a list of elements (list1) and a search element.

Step 3: Initialize: A variable found = False to track whether the element is found.

Step 4: Iterate over the list using a loop:

For each element in list1, check:

If the current element is equal to search:

Set found = True.

Print the output as "Element found at position <index>" (index starts from 1).

Break the loop.

Step 5: Check:

If found == False:

Print the output as "Element not found".

Step 6: End

PROGRAM:

```
list1=[4,1,2,5,3]
search=int(input("Enter the Search Number:"))
found=False
for i in range(0,len(list1)):
    if search==list1[i]:
        print("%d Number is Found in the Position:%d" %(search,i))
        found=True
if found==False:
    print("%d Number is Not Found in the List" %(search))
```

OUTPUT:

Enter the Search Number: 5

5 Number is Found in the Position: 3

RESULT:

Thus the program for linear search is written and executed successfully.

EX NO: 6 B **BINARY SEARCH**

AIM:

Write a program to search an element in a list using binary search concept in python.

ALGORITHM:

Step 1: Start

Step 2: Input a sorted list arr and the target element target.

Step 3: Initialize:

first = 0 (index of the first element in the list)

last = len(arr) - 1 (index of the last element in the list)

found = False (to track if the element is found)

Step 4: Repeat while first <= last and found == False:

Calculate the middle index: mid = (first + last) // 2.

Check:

If arr[mid] == target:

Set found = True.

Print: "Element found at position <mid>" (position starts from 0).

Break the loop.

Else if target < arr[mid]:

Update last = mid - 1 (search in the left half).

Else:

Update first = mid + 1 (search in the right half).

Check:

If found == False:

Print "Element not found".

Step 5:End

PROGRAM:

```
def binary_search(item_list,item):
    first = 0
    last = len(item_list)-1
    found = False
while( first<=last and not found):
```

```
        mid = (first + last)//2
        if item_list[mid] == item :
          found = True
          print("Element found in the position:",mid)
        else:
          if item < item_list[mid]:
            last = mid - 1
          else:
            first = mid + 1
    if (found == False):
       print("Element not found")
a=[]
n=int(input("Enter the number of elements:"))
for i in range(0,n):
   b=int(input("Enter the element:"))
   a.append(b)
x=int(input("Enter the number to be searched:"))
binary_search(a,x)
```

OUTPUT:

```
Enter the number of elements: 5
Enter the element: 10
Enter the element: 20
Enter the element: 30
Enter the element: 40
Enter the element: 50
Enter the number to be searched: 30
Element found in the position:2
```

RESULT:

Thus the program for binary search is written and executed successfully.

EX NO: 7 READ THE CONTENT OF THE FILE AND CHANGE THEM FROM LOWER TO UPPER CASE CHARACTERS

AIM:

To read the contents of the file and change them from lower to upper case characters in python.

ALGORITHM:

Step 1: Start the program.

Step 2: Open the file in read mode

Step 3: Read each line to calculate the word count.

```
for line in f:
    print(line.upper())
```

Step 4: Print the line

Step 5: Stop the program.

PROGRAM:

```
fname = input("Enter file name: ") #test.txt
with open(fname, 'r') as f:
    for line in f:
        print (line.upper())
```

Create another file:test.txt

```
hai
hello python
hai friends
```

OUTPUT:

Enter file name: “test.txt”

```
HAI
HELLO PYTHON
HAI FRIENDS
```

RESULT:

Thus the program to change the file from lower to upper case has been executed successfully.

EX NO: 8 **COMMAND LINE ARGUMENTS (WORD COUNT)**

AIM:

To count the words in the text file using the command line arguments in python programming.

ALGORITHM:

Step 1: Start the program.

Step 2: Open the file in read mode

Step 3: Read each line to calculate the word count.

```
for line in f:
    words = line.split()
    num_words += len(words)
```

Step 4: Print the number of words

Step 5: Stop the program.

PROGRAM:

```
fname = input("Enter file name: ") #test.txt
num_words = 0
with open(fname, 'r') as f:
    for line in f:
        words = line.split()
        num_words += len(words)
print("Number of words:")
print(num_words)
```

Create another file:test.txt

```
hai
hello python
hai friends
```

OUTPUT:

Enter file name: “test.txt”

Number of words:

5

RESULT:

Thus the program for word count in file has been executed successfully.

EX NO: 9 **FIND THE MOST FREQUENT WORDS IN A TEXT READ FROM A FILE**

AIM:

To find the most frequent words in a text file using python programming.

4

ALGORITHM:

Step 1: Start the program.

Step 2: import the module counter from package collections

Step 3: Open and read the file and increment the counter for each word

Step 4: Print the number of words and its counter value

Step 5: Stop the program.

PROGRAM:

```
import sys
filename = sys.argv[1]
with open(filename, “r”) as f:
  text = f.read() words = text.split()
 print(“Word count:”, len(words))
```

Create another file:test.txt

```
hai
hello python
hai friends
```

OUTPUT:

```
number of words in the file:
Counter({'hai': 2, 'friends': 1, 'hello': 1, ‘python’: 1}))
```

RESULT:

Thus the program for most frequent words in a text file has been executed successfully.

EX NO: 10A **STUDENT MARKS STATEMENT USING DICTIONAY**

AIM:

To generate student marks statement using dictionary in python programming.

ALGORITHM:

Step 1: Start

Step 2:Input:

Student's name and store it as student['Name'].

Student's roll number and store it as student['Roll Number'].

Step 3: Input the number of subjects num_subjects.

Step 4: Initialize:

An empty dictionary student['Marks'] to store subject-wise marks.

Step 5: Repeat for each subject (from 1 to num_subjects):

Input the subject name as subject.

Input the marks for the subject as marks.

Store marks in student['Marks'] with subject as the key.

Step 6: Compute:

Total marks as the sum of all marks in student['Marks'].

Average marks as total / num_subjects.

Step7: Print the output:

Student's name, roll number, and subject-wise marks.

Total marks.

Average marks rounded to two decimal places.

If average >= 50, Output: "Result: Passed".

Else, Output: "Result: Failed".

Step 8:End

PROGRAM:

```
def display_marks_statement(student):
    print("\n--- Student Marks Statement ---")
    print(f"Name: {student['Name']}")
    print(f"Roll Number: {student['Roll Number']}")
```

```
    print("Subject-wise Marks:")
    for subject, marks in student['Marks'].items():
        print(f"  {subject}: {marks}")
    total = sum(student['Marks'].values())
    average = total / len(student['Marks'])
    print(f"Total Marks: {total}")
    print(f"Average Marks: {average:.2f}")
    if average >= 50:
        print("Result: Passed")
    else:
        print("Result: Failed")

# Input student information and marks
student= {}
student['Name'] = input("Enter the student's name: ")
student['Roll Number'] = input("Enter the roll number: ")
num_subjects = int(input("Enter the number of subjects: "))
student['Marks'] = {}

for _ in range(num_subjects):
    subject = input("Enter the subject name: ")
    marks = int(input(f"Enter the marks for {subject}: "))
    student['Marks'][subject] = marks

# Display the marks statement
display_marks_statement(student)
```

OUTPUT:

```
Enter the student's name: John Doe
Enter the roll number: 101
Enter the number of subjects: 3
Enter the subject name: Math
Enter the marks for Math: 85
Enter the subject name: Science
Enter the marks for Science: 90
```

```
Enter the subject name: English
Enter the marks for English: 75

--- Student Marks Statement ---
Name: John Doe
Roll Number: 101
Subject-wise Marks:
  Math: 85
  Science: 90
  English: 75
Total Marks: 250
Average Marks: 83.33
Result: Passed
```

RESULT:

Thus student marks statement using dictionary in python programming has been executed successfully.

Ex NO: 10B **RETAIL BILL PREPARATIONUSING DICTIONAY**

AIM:

To generate retail bill statement using dictionary in python programming.

ALGORITHM:

Step 1: Start

Step 2: Input the number of items num_items.

Step 3: Initialize an empty dictionary items to store item details.

Step 4: Repeat for each item (from 1 to num_items):

Input the item name as item_name.

Input the quantity for the item as quantity.

Input the price per unit for the item as price_per_unit.

Store the item details in the dictionary items with item_name as the key and a nested dictionary containing:

'Quantity': the value of quantity.

'Price per Unit': the value of price_per_unit.

Step 5: Initialize a variable total_amount = 0 to keep track of the total bill amount.

Step 6: Output:

Print the header for the bill: "Item Name", "Quantity", "Price per Unit", and "Total Price".

Step 7: For each item in the dictionary:

Retrieve the item name, quantity, and price per unit.

Calculate the total price for the item as: total_price = quantity * price_per_unit.

Output the item details in a formatted way:

Item name, quantity, price per unit, and the calculated total price.

Add the total_price to total_amount for the overall bill.

Step 8: Output the total amount due:

Print the total amount of the bill.

Step 9: End

PROGRAM:

```
def generate_bill(item_dict):
    print("\n--- Retail Bill Statement ---")
    print(f"{'Item Name':<20} {'Quantity':<10} {'Price per Unit':<15} {'Total Price':<15}")
    total_amount = 0

    # Loop through the dictionary and print the bill details for each item
    for item, details in item_dict.items():
        item_name = item
        quantity = details['Quantity']
        price_per_unit = details['Price per Unit']
        total_price = quantity * price_per_unit

        print(f"{item_name:<20} {quantity:<10} {price_per_unit:<15} {total_price:<15}")

        total_amount += total_price

    # Print the total amount
    print(f"\n{'Total Amount':<20} {'':<10} {'':<15} {total_amount:<15}")


# Input items and their details
items = {}
num_items = int(input("Enter the number of items: "))

for _ in range(num_items):
    item_name = input("Enter the item name: ")
    quantity = int(input(f"Enter the quantity for {item_name}: "))
    price_per_unit = float(input(f"Enter the price per unit for {item_name}: "))

    # Store item details in the dictionary
    items[item_name] = {
        'Quantity': quantity,
        'Price per Unit': price_per_unit
    }

# Generate and display the retail bill
```

```
generate_bill(items)
```

OUTPUT:

```
Enter the number of items: 3
Enter the item name: Apple
Enter the quantity for Apple: 2
Enter the price per unit for Apple: 1.5
Enter the item name: Banana
Enter the quantity for Banana: 5
Enter the price per unit for Banana: 0.8
Enter the item name: Orange
Enter the quantity for Orange: 3
Enter the price per unit for Orange: 1.2

--- Retail Bill Statement ---
Item Name        Quantity  Price per Unit  Total Price
Apple            2         1.5             3.0
Banana           5         0.8             4.0
Orange           3         1.2             3.6

Total Amount     12.6
```

RESULT:

Thus student marks statement using dictionary in python programming has been executed successfully.

EX NO: 11 **PROGRAM USING SETS TO COMPUTE INTERSECTION, UNION AND SYMMETRIC DIFFERENCE BETWEEN SETS.**

AIM:

To write a python program using sets to compute intersection, union and difference between sets.

ALGORITHM:

Step1: Start

Step 2: Input the elements of the first set (set1):

Convert the input string into a set of integers.

Step 3: Input the elements of the second set (set2):

Convert the input string into a set of integers.

Step 4: Compute Intersection:

Find the intersection of set1 and set2 using the & operator.

Store the result in a variable intersection.

Step 5: Compute Union:

Find the union of set1 and set2 using the | operator.

Store the result in a variable union.

Step 6: Compute Symmetric Difference:

Find the symmetric difference of set1 and set2 using the ^ operator.

Store the result in a variable symmetric_difference.

Step 7: Output:

Print the intersection result.

Print the union result.

Print the symmetric_difference result.

Step 8: End

PROGRAM:

```
def compute_intersection(set1, set2):
    # Compute intersection of set1 and set2
    return set1 & set2

def compute_union(set1, set2):
    # Compute union of set1 and set2
```

```
    return set1 | set2

def compute_symmetric_difference(set1, set2):
    # Compute symmetric difference between set1 and set2
    return set1 ^ set2

# Input two sets from the user
set1 = set(map(int, input("Enter the elements of set 1 (separated by space): ").split()))
set2 = set(map(int, input("Enter the elements of set 2 (separated by space): ").split()))

# Compute and display the results
intersection = compute_intersection(set1, set2)
union = compute_union(set1, set2)
symmetric_difference = compute_symmetric_difference(set1, set2)

# Output the results
print("\nIntersection of set 1 and set 2:", intersection)
print("Union of set 1 and set 2:", union)
print("Symmetric Difference between set 1 and set 2:", symmetric_difference)
```

OUTPUT:

```
Enter the elements of set 1 (separated by space): 1 2 3 4 5
Enter the elements of set 2 (separated by space): 4 5 6 7

Intersection of set 1 and set 2: {4, 5}
Union of set 1 and set 2: {1, 2, 3, 4, 5, 6, 7}
Symmetric Difference between set 1 and set 2: {1, 2, 3, 6, 7}
```

RESULT:

Thus a python program using sets to compute intersection, union and difference between sets has been executed successfully.

EX NO: 12A **Voter's age validation using Exception**

AIM:

To write a python program to validate the voter's age using exception.

ALGORITHM:

Step 1: Start

Step 2: Define a custom exception InvalidAgeException:

Create a class that inherits from the Exception class.

The constructor should accept a message and pass it to the Exception class.

Step 3: Define the function validate_voter_age(age):

Input: age (the age of the voter).

If age < 18:

Raise an InvalidAgeException with a message "Age is below 18, not eligible to vote."

Else:

Print "You are eligible to vote."

Step 4: Main program:

Input the user's age and store it in the variable age.

Try block:

Call the function validate_voter_age(age) to validate the age.

Step 5: Exception Handling:

If InvalidAgeException is raised:

Print the error message from the exception (e.message).

If ValueError occurs (non-integer input):

Print "Invalid input! Please enter a valid integer for age."

Step 6: End

PROGRAM:

```
class InvalidAgeException(Exception):
    """Custom exception for invalid age."""
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)
```

```
def validate_voter_age(age):
    # Check if the age is valid for voting (18 or older)
    if age < 18:
        raise InvalidAgeException("Age is below 18, not eligible to vote.")
    else:
        print("You are eligible to vote.")

# Main program
try:
    # Input age from the user
    age = int(input("Enter your age: "))

    # Validate the voter age
    validate_voter_age(age)

except InvalidAgeException as e:
    print(f"Error: {e}")

except ValueError:
    print("Invalid input! Please enter a valid integer for age.")
```

OUTPUT:

```
Enter your age: 16
Error: Age is below 18, not eligible to vote.


Enter your age: 20
You are eligible to vote.
```

RESULT:

Thus a python program to validate the voter's age using exception has been executed successfully.

EX NO: 12B **MARKS RANGE VALIDATION USING EXCEPTION**

AIM:

To write a python program to validate the marks using exception.

ALGORITHM:

Step 1: Start

Step 2: Define a custom exception InvalidMarksException:

Create a class InvalidMarksException that inherits from the Exception class.

The constructor should accept a message and pass it to the Exception class.

Step 3: Define the function validate_marks(marks):

Input: marks (integer representing the marks entered by the user).

If marks is less than 0 or greater than 100:

Raise the InvalidMarksException with the message "Marks must be between 0 and 100."

Else:

Print the message marks entered is valid.

Step 4: Main program:

Input: Get the user's marks and store it in the variable marks.

Try block:

Call the validate_marks(marks) function to validate the marks.

Step 5: Exception Handling:

If InvalidMarksException is raised:

Catch the exception and print the error message from the exception.

If ValueError occurs (when the user enters a non-integer value):

Catch the exception and print "Invalid input! Please enter a valid integer for marks."

Step 6: End

PROGRAM:

```
class InvalidMarksException(Exception):
    """Custom exception for invalid marks."""
    def __init__(self, message):
        self.message = message
        super().__init__(self.message)
```

```
def validate_marks(marks):
    # Validate that marks are within the range 0 to 100
    if marks < 0 or marks > 100:
        raise InvalidMarksException("Marks must be between 0 and 100.")
    else:
        print(f"Marks entered: {marks} - Valid marks.")

# Main program
try:
    # Input marks from the user
    marks = int(input("Enter marks: "))

    # Validate the marks
    validate_marks(marks)

except InvalidMarksException as e:
    print(f"Error: {e}")

except ValueError:
    print("Invalid input! Please enter a valid integer for marks.")
```

OUTPUT:

```
Enter marks: 105
Error: Marks must be between 0 and 100.
```

```
Enter marks: 85

Marks entered: 85 - Valid marks.
```

RESULT:

Thus a python program to validate the marks using exception has been executed successfully.

EX NO: 13 **STUDENT INFORMATION USING CLASS AND OBJECT**

AIM:

To write a python program for displaying student information using class and object.

ALGORITHM:

Step 1: **Start**

Step 2: **Define a class Student**:

Step 3: Define the constructor __init__():

Input: Name, roll number, age, course.

Initialize the following attributes:

- name = input name
- roll_number = input roll number
- age = input age
- course = input course

Step 4: **Define a method display_info()** in the class Student:

Output the student's information:

- Print "Name: {self.name}"
- Print "Roll Number: {self.roll_number}"
- Print "Age: {self.age}"
- Print "Course: {self.course}"

Step 5: **Create objects** of the class Student:

Object student1 with specified details (e.g., name, roll number, age, course).

Object student2 with specified details (e.g., name, roll number, age, course).

Step 6: **Call display_info() method** for each object:

Call student1.display_info() to display information of student1.

Call student2.display_info() to display information of student2.

Step 7: **End**

PROGRAM:

```
class Student:
    # Constructor to initialize student information
    def __init__(self, name, roll_number, age, course):
        self.name = name
        self.roll_number = roll_number
        self.age = age
        self.course = course

    # Method to display student information
    def display_info(self):
        print("\n--- Student Information ---")
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Age: {self.age}")
        print(f"Course: {self.course}")

# Create student objects and input details
student1 = Student("John Doe", "101", 20, "Computer Science")
student2 = Student("Jane Smith", "102", 21, "Electrical Engineering")

# Display student information
student1.display_info()
student2.display_info()
```

OUTPUT:

```
--- Student Information ---
Name: John Doe
Roll Number: 101
Age: 20
```

Course: Computer Science

--- Student Information ---
Name: Jane Smith
Roll Number: 102
Age: 21
Course: Electrical Engineering

RESULT:

Thus a python program for displaying student information using class and object has been executed successfully.

EX NO: 14 **DEPOSIT OR WITHDRAW MONEY IN A BANK ACCOUNT USING CLASS AND OBJECTS**

AIM:

To write a python program for displaying bank account details such as deposit or withdraw using class and object.

ALGORITHM:

Step 1: Start

Step 2: Define the BankAccount class:

Constructor __init__(self, account_holder, balance=0):

Initialize the account holder's name (account_holder).

Initialize the balance (balance) of the account (default is 0).

Step 3: Define the deposit(self, amount) method:

If the amount is greater than 0:

Add amount to the balance.

Print the updated balance.

Else:

Print an error message indicating the deposit amount should be positive.

Step 4: Define the withdraw(self, amount) method:

If the amount is less than or equal to the balance and greater than 0:

Subtract amount from the balance.

Print the updated balance.

Else:

Print an error message indicating insufficient balance or invalid withdrawal amount.

Step 5: Define the display_account_info(self) method:

Print the account holder's name and current balance.

Step 6: Create an object of the BankAccount class with an initial balance (e.g., account1 = BankAccount("John Doe", 1000)).

Step 7: Call display_account_info() to show the account details.

Step 8: Perform deposit operation:

Call deposit() with the deposit amount.

Step 9: Perform withdrawal operation:

Call withdraw() with the withdrawal amount.

Step 10: Call display_account_info() again to show the updated account details after transactions.

Step 11: End

PROGRAM:

```
class BankAccount:
    # Constructor to initialize account details
    def __init__(self, account_holder, balance=0):
        self.account_holder = account_holder
        self.balance = balance

    # Method to deposit money into the account
    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: {amount}. New balance: {self.balance}")
        else:
            print("Deposit amount should be positive.")

    # Method to withdraw money from the account
    def withdraw(self, amount):
        if amount <= self.balance and amount > 0:
            self.balance -= amount
            print(f"Withdrawn: {amount}. New balance: {self.balance}")
        else:
            print("Insufficient balance or invalid withdrawal amount.")

    # Method to display account details
    def display_account_info(self):
        print("\n--- Account Information ---")
        print(f"Account Holder: {self.account_holder}")
        print(f"Current Balance: {self.balance}")

# Creating an object of the BankAccount class
account1 = BankAccount("John Doe", 1000)
```

```
# Display account details
account1.display_account_info()

# Deposit money into the account
account1.deposit(500)

# Withdraw money from the account
account1.withdraw(200)

# Display account details after transactions
account1.display_account_info()
```

OUTPUT:

```
--- Account Information ---
Account Holder: John Doe
Current Balance: 1000

Deposited: 500. New balance: 1500
Withdrawn: 200. New balance: 1300

--- Account Information ---
Account Holder: John Doe
Current Balance: 1300
```

RESULT:

Thus a python program for displaying student information using class and object has been executed successfully.